

第 8 章

软件本地化测试

随着软件市场越来越趋向于全球化的竞争,为了使将来软件产品可以走向世界,能够参与全球市场的竞争,在开发软件产品的时候就需要考虑到如何适应国际化的需求、满足不同国家或地区的用户使用要求,包括不同语言、不同货币、不同计量单位和不同文化习俗等方面对软件产品所提出的要求,这就产生了软件国际化和本地化的概念。

本章主要介绍什么是软件本地化、软件本地化的翻译验证和其他测试重点,让读者深入了解软件本地化的过程,并全面了解如何完成本地化测试。

8.1 什么是软件本地化

软件本地化是将一个软件产品按特定国家或语言市场的需要进行全面定制的过程,包括翻译、重新设计、功能调整以及功能测试、是否符合各个地方的习俗、文化背景、语言和方言的验证等。在开始讨论之前,先来介绍几个关键术语。

(1) L10n: 英文 Localization 一词的简写,意即本地化,由于首字母“L”和末尾字母“n”间有 10 个字母,所以简称 L10n。

(2) I18n: 英文 Internationalization 的简写,意为国际化,由于首字母“I”和末尾字母“n”间有 18 个字符,所以简称 I18n。Internationalization 指为保证所开发的软件能适应全球市场的本地化工作而不需要对程序做任何系统性或结构性变化的特性,这种特性通过特定的系统设计、程序设计、编码方法来实现。也就是说,完全符合国际化的软件产品,在对其进行本地化工作的时候,只要进行一些配置和翻译工作,而不需要修改软件的程序代码。

(3) locale: 场所、本地,简单来说是指语言和区域进行特殊组合的一个标志。

(4) Globalization: 即全球化,是一个概念化产品的过程,它基于全球市场考虑,以便一个产品只做较小的改动就可以在世界各地出售。全球化可以看作国际化和本地化两者合成的结果。

它们之间的关系可用图 8-1 表示,在这里强调国际化是核心工作,

只有满足国际化的要求之后才能容易实现本地化,而且翻译只是本地化工作的一部分,全球化是一个产品市场的概念。提到本地化,首先想到的就是翻译问题,毋庸置疑,翻译在本地化工作中占据着很重要的地位,但是绝不能把翻译等同于本地化,它和本地化还有很大的差距。当文字被翻译后,还要对产品进行其他相应的更改,这些更改包括技术层面和文化层面的更改。

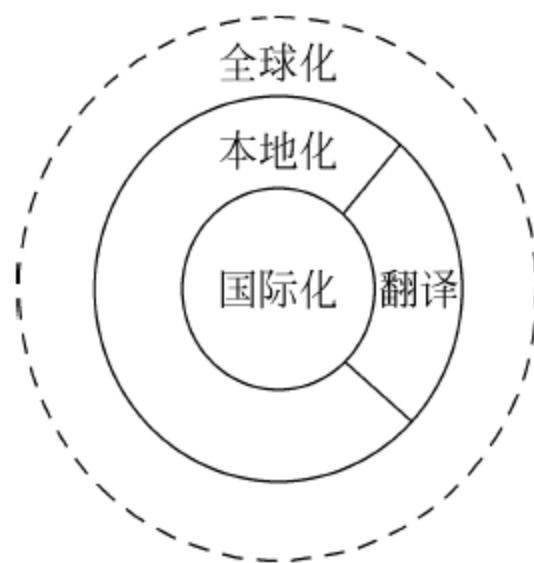


图 8-1 翻译、本地化与国际化、全球化之间的关系

8.1.1 软件本地化与国际化

人们常说的“国际化”是指产品走出国门,在其他国家销售。但在软件产品开发中,产品国际化有着不同的含义,意味着对软件“原始产品”本地化的支持,也就是为了解决软件能在各种不同语言、不同风俗的国家和地区使用的问题,对计算机设计和编程所做出的某些规定。为了减少本地化的工作,软件产品国际化应该具有下面一系列特性。

- (1) 支持 Unicode 字符集;
- (2) 分离程序代码和显示内容(文本、图片、对话框、信息框和按钮等),如将这些内容由资源文件(如 *.rc、.properties)统一处理;
- (3) 消除硬代码(Hard Code,指程序代码中所包含一些特定的数据,它们本应该作为变量处理,而对应的具体数据应该存储在数据库或初始化文件中);
- (4) 使用 Header Files 去定义经常被调用的代码段;
- (5) 改善翻译文本尺寸,具有调整的灵活性,如在资源文件中可以直接具有调整用户界面的灵活性来适应翻译文本尺寸;
- (6) 支持各个国家的键盘设置,并有对应的热键处理;
- (7) 支持文字不同方向的显示;
- (8) 支持各个国家的度量衡、时区、货币单位格式等自定义功能;
- (9) 用户界面(包括颜色、字体)等自定义特性。

软件本地化是国际化向特定本地语言环境的转换,即将软件从源语言转换成一种或多种目标语言的过程,同时针对目标国家或地区,对产品的外观、参数设置等进行相应的处理,如:

- (1) 软件用户界面(User Interface,UI)默认值的设置;
- (2) 联机文档(帮助文档、技术支持站点等);
- (3) 数据库初始化工作;
- (4) 热键设置;
- (5) 度量衡和时区等。

国际化与本地化是一个辨证的关系,本地化要适应国际化的规定。而国际化是本地化的基础和前提,为本地化做准备,使本地化过程不需要对代码做改动就能完成,或将代码修改降到最低限度。

8.1.2 字符集问题

要支持软件国际化特征,首先就要考虑使用正确的字符集。西方语言,如英语、法语和德语,使用不到 256 个字符,所以它们可以用单字节编码表示。可是亚洲语言,比如日文和中文,

却有几万个字符,因此需要双字节编码。所以在做本地化测试的时候,应该检查开发人员是否使用了正确的字符编码。

字符集是操作系统中所使用的字符映射表,例如,早期的 UNIX 系统使用只包含 128 个字符的 7-bit ASCII 字符集(包括 Tabs、空格、标点、符号、大小写字母、数字和回车键等)。然而对于很多语言来说,7-bit ASCII 字符集远远不够,因为它不包含特殊字符(比如 é、å 或 â)。所以后来出现了 8-bit ASCII,它包含 256 个字符。微软的 Windows 早期版本使用 8-bit ASCII 字符集,对于 UNIX 计算机,还有一个 ISO 标准(ISO 8859-X),和 8-bit ASCII 相似。即使拥有 256 个字符,8-bit ASCII 还是无法满足所有语言的需求。汉语、日语和韩语这些语言的字符都很多,无法适用扩展后的 ASCII 字符集,对于这些语言,可以使用 16-bit 字符集(双字节、多字节或变数字字节),这就是统一的字符编码标准 Unicode,采用双字节对字符进行编码,几乎包含所有语言的每个字符。

Unicode 是一个国际标准(<http://www.unicode.org/standard/standard.html>),采用双字节对字符进行编码,提供了在世界主要语言中通用的字符,所以也称为基本多文种平面。Unicode 以明确的方式表述文本数据,简化了混合平台环境中的数据共享。目前,很多操作系统都支持 Unicode,包括 Windows 系统、Linux 系统和 Mac OS、Solaris、IBM-AIX、HP-UX 等。Unicode 简称为 UCS,现在用的是 UCS-2,即 2 字节编码,和国际标准字符集 ISO 10646-1 相对应。UCS 最新版本是 2005 年的 Unicode 4.1.0,而 ISO 的最新标准是 ISO 10646-3:2003。

UCS 只是规定如何编码,并没有规定如何传输、保存编码。所以有了 Unicode 实用的编码体系,如 UTF-8、UTF-7、UTF-16。UTF-8(UCS Transformation Format)和 ISO-8859-1 完全兼容,解决了 Unicode 编码在不同的计算机之间的传输、保存,使得双字节的 Unicode 能够在现存的处理单字节的系统上正确传输。UTF-8 使用可变长度字节来储存 Unicode 字符,这能解决敏感字符引起的问题。前面有几个 1,表示整个 UTF-8 串是由几个字节构成的。以下是 Unicode 和 UTF-8 之间的转换关系:

```
U - 00000000 - U - 0000007F: 0xxxxxxx
U - 00000080 - U - 000007FF: 110xxxxx 10xxxxxx
U - 00000800 - U - 0000FFFF: 1110xxxx 10xxxxxx 10xxxxxx
U - 00010000 - U - 001FFFFF: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
U - 00200000 - U - 03FFFFFF: 111110xx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx
U - 04000000 - U - 7FFFFFFF: 1111110x 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx
```

8.1.3 软件国际化标准

软件达到什么样的程度才算彻底实现了国际化?虽然在这上面,仍然存在一定的分歧,但普遍认为作为国际化软件,要么在应用软件运行时可以动态切换某种国家或地区的语言,要么在应用软件启动前或启动时可以设置某种语言。例如,操作系统 Windows XP,不需要重新编译,就可以切换到不同语言和不同的国家或地区。作为国际化软件的规范可以归纳为以下 5 点。

- (1) 切换语言的机制。
- (2) 与语言无关的输出接口。
- (3) 与语言无关的输入接口和标准的输入协议。
- (4) 资源文件的国际化。

(5) 支持和包容本地化数据格式。

为了使软件国际化更为规范,需要建立相应的国际标准,来规范字符集、编码、数据交换、语言输入方法、输出(打印、用户界面)、字体处理、文化习俗等各个方面。比较著名的一些国际化标准组织有:

- (1) ANSI(American National Standards Institute)
- (2) POSIX(Portable Operating System Interface for Computer Environments)
- (3) ISO(International Standards Organization)
- (4) IEEE(Institute of Electrical and Electronics Engineers)
- (5) Unicode Consortium
- (6) Open Group(X Consortium and OSF)
- (7) Linux(Linux International),X/Open and XPG

而与国际化有密切关系的国际标准有:

- (1) ISO/IEC 10646-1:2003 定义了 4 字节编码的通用字符集(Universal Character Set, UCS),也称通用多 8 位编码字符集(Universal Multiple-Octet Coded Character Set)。
- (2) ISO 639-1:2002,2 字母语种代码(alpha-2)标准。
- (3) ISO 3166-1:1997,国家代码标准。
- (4) RFC 3066,语言鉴定标签标准。

8.1.4 软件本地化基本步骤

要做好软件本地化的测试工作,有必要了解软件本地化的步骤。软件本地化的基本工作是建立在软件国际化的基础上,或者说,软件本地化的第一项工作就是规范甚至是迫使源语言版本的开发遵守软件国际化的标准。在此基础上,依次做好版本管理、建立专业术语表、翻译、调整 UI 等工作。

在软件全部翻译完毕,对技术部分做了必要的调整之后,软件产品或多或少发生了一些变化。不论原来的软件产品有多成熟,经过本地化工作之后,软件产品可能产生一些新的问题,或者引起一些回归缺陷,所以针对本地化的产品进行测试,也是必要的。以下是本地化的基本步骤,虽然在具体操作时可能会有所不同,但基本步骤是不可省略的。

- (1) 建立一个配置管理体系,跟踪目标语言各个版本的源代码;
- (2) 创造和维护术语表;
- (3) 从源语言代码中分离资源文件或提取需要本地化的文本;
- (4) 把分离或提取的文本、图片等翻译成目标语言;
- (5) 把翻译好的文本、图片重新插入目标语言的源代码版本中;
- (6) 如果需要,编译目标语言的源代码;
- (7) 测试翻译后的软件,调整 UI 以适应翻译后的文本;
- (8) 测试本地化后的软件,确保格式和内容都正确。

本地化的工作流程如图 8-2 所示,其中 DTP 指多语言桌面排版(Multilingual Desktop Publishing)。本地化领域的桌面出版,是指将采用某一语言的原始文档(如操作手册、产品样本、宣传单页等)按照一种或多种目标语言重新排版,形成不同的语言版本。一些产品可能支持或不支持对某种语言的拼写检查,而需要特定的操作系统,例如,针对 Macintosh 计算机的日语工具箱或针对 PC 的阿拉伯视窗。

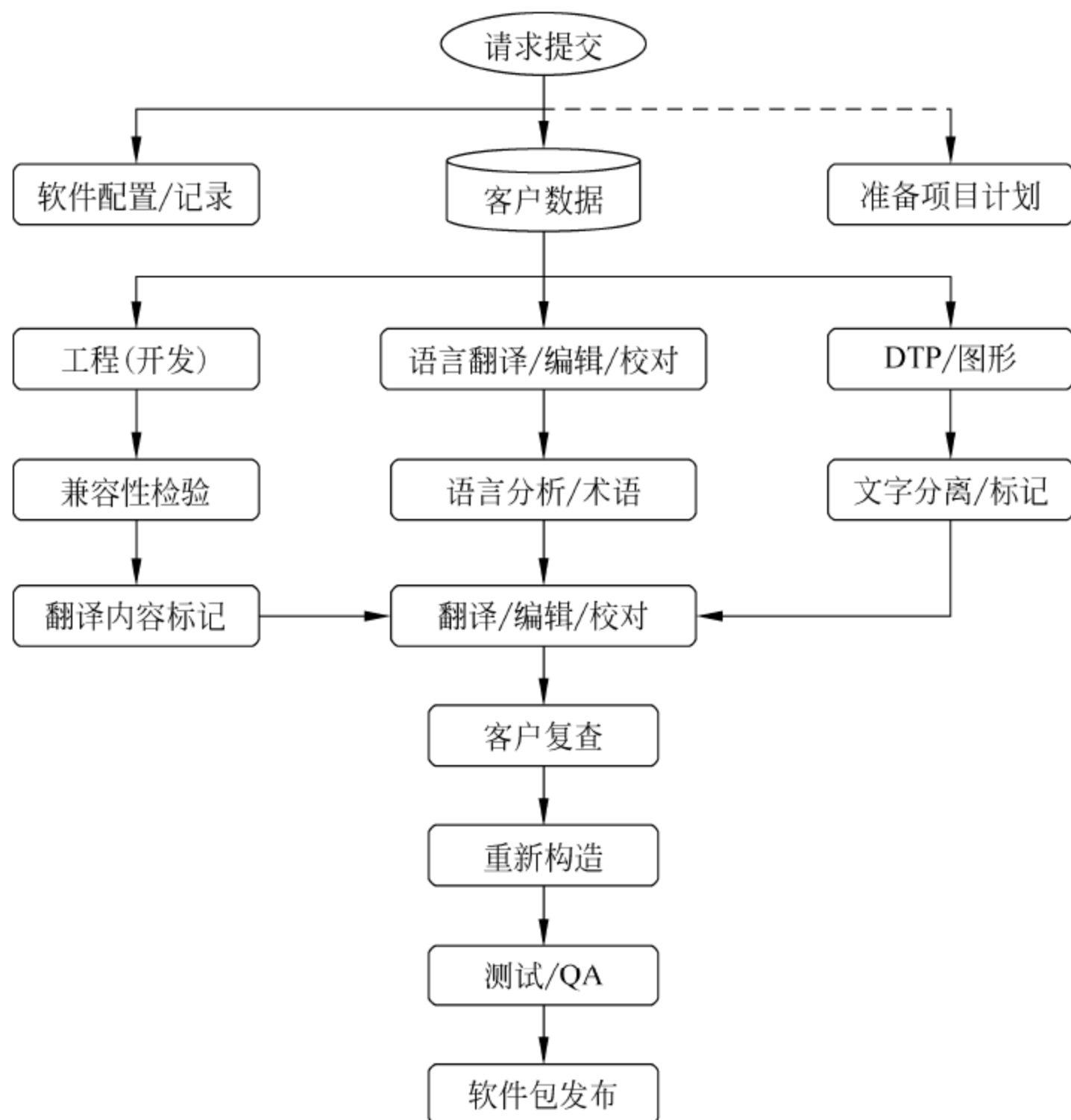


图 8-2 软件本地化的工作流程

8.1.5 软件本地化测试

在进行软件本地化测试之前,先要检查软件源语言开发是否遵守了软件国际化方面的规范,验证是否具有软件国际化所应有的全部特征,包括字符集、资源和代码分离、时区设置、语言和地方选择等。要验证软件是否具备国际化特征,需要根据软件国际化相关标准进行评审,包括设计和代码的评审。

仅仅评审是不够的,还要进行相关的功能测试、界面测试,但不包括翻译验证等。而功能测试和界面测试,一般采用伪翻译(Pseudocode, Pseudo-translation)所构建的版本进行测试。这种 Pseudocode 版本是将文字、图片信息中的源语言被混合式多种语言(如英文、中文、日文和德文等)替代而构建的一种临时的、专供测试用的版本。

在源语言版本通过了国际化验证之后,才开始进行本地化的测试。本地化测试的版本,就是待发布的特定语言的真实版本,而该特定的语言版本是在源语言版本的基础上,经过本地化工作之后而获得的。本地化测试着重于以下几个方面。

(1) 主要的功能性测试,函数之间传递的参数、数据库的默认值经过本地化处理后,可能会对系统的功能运行产生较大的影响,从而引起功能缺陷;

(2) 在本地化环境中的安装和升级测试,由于目标语言的操作系统和软件本身都不一样了,安装或升级过程也常常受到影响;

(3) 根据产品的目标区域而进行的应用程序和硬件兼容性测试,其应用程序的接口、标准可能不同,硬件型号及其配置更有可能存在差异;

(4) 受本地化影响的用户界面,包括布局、格式、文字和图片等内容显示问题;

(5) 特殊的语言环境(意境)、文化背景和地理位置等可能给软件带来的问题;

(6) 文字翻译的正确性、准确性以及是否遗漏等。

具体测试的时候,针对上述各项内容,还要进一步细化,确定具体的测试需求,例如,针对用户界面和语言文化方面的测试,其具体内容包括以下几个方面。

(1) 应用程序源文件的有效性;

(2) 验证语言的准确性和源代码的属性;

(3) 排版错误;

(4) 检查印刷文档和联机帮助、界面信息的一致性以及命令键的顺序等;

(5) 用户界面是否符合当地审美标准(或情趣);

(6) 文化适用性的评估;

(7) 政治敏感内容的检查。

当发布一个本地化产品时,应该确保本地化文档(用户手册、在线帮助、帮助文件等)都包含在其中。同时应该检查翻译的质量和完整性,并确保所有的文档和应用程序界面中术语使用的一致性。所以概括起来,本地化测试包括以下 6 个方面。

(1) 功能性测试,所有基本功能、安装、升级等测试;

(2) 翻译测试,包括语言完整性、术语准确性等检查;

(3) 可用性测试,包括用户界面、度量衡和时区等适合当地的要求;

(4) 兼容性测试,包括硬件软件本身、第三方软件兼容性等的测试;

(5) 文化、宗教、喜好等适用性测试;

(6) 手册验证,包括联机文件、在线帮助、PDF 文件等测试。

由此可见,整个软件本地化的过程,其实是一个再创造的过程。文字翻译只做了本地化工作的一部分,要真正完成软件本地化确实有很多工作要做。

8.2 翻译验证

软件本地化其中一项重要的工作就是文字翻译,主要任务是把源语言转换到另一种目标语言,而与此对应的就是翻译验证。如前所述,本地化不仅是简单的文字翻译转换,还应该根据目标语言国家的市场特点、文化习惯、法律等情况进行本地特性开发、界面布局调整等工作。所以翻译也不是单纯的翻译,还必须立足于文化和市场的角度来考虑用户,兼顾目标语言的文化心理。

翻译验证,需要对翻译内容、语言文化以及特殊符号等进行检查,帮助翻译人员发现翻译中的错误和不妥之处、指出开发人员技术上未能实现的部分。例如,把一种语言翻译成另外一种语言,难免会有晦涩或表达不准确的地方,由于大量的翻译工作,翻译人员可能会遗漏或疏忽某些地方,这就需要测试人员进行检查和验证。如果有条件,在本地化软件向市场发布之前,还应该让目标语言的语言专家来最后审稿。

1. 内容的验证

一般来说,需要翻译的内容大致分为三个部分:用户界面、联机文档和用户手册等。首先,从源代码中或直接从资源文件中把需要翻译的文字、图片提取出来,存储在相应的数据库或本地化管理系统中,补充版本、文件名和位置等信息。然后,将要翻译的素材交给本地化团队去翻译或由第三方专业翻译公司去翻译。将所翻译的结果进行保存,同时处理,构建相应的

资源文件或软件包,即创建新的语言版本。这个阶段的要求是翻译准确,能够照顾到目标语言的文化和习惯,并能完成不同语言版本的自动构建。

我们知道,翻译是本地化的一项重要工作,不同的语言使用不同的语法和句子结构,如英语、汉语等基本句子结构是“主+谓+宾”,但日语的基本句子结构是“主+宾+谓”,所以词对词的直译往往是不行的,还必须考虑上下文关系,根据程序中特定的语境来决定文字的含义,才能保证翻译准确。

在保持原有意思和风格的基础上,还必须把源语言格式替换为目标语言的格式。例如,联机文档常用的格式有 HTML、PDF 和 CHM 文件等,本地化翻译人员也应该把翻译后的文档转换成相应的格式。测试人员也要注意其转换后的格式是否能够正常显示,各部分内容和相关的链接是否正常等。

除此之外,软件中的按钮、图标和插图等上面的文字也需要翻译,本地化测试人员应该指出翻译人员没有翻译的部分,协助其尽快完成。关于翻译验证,有以下几点建议。

- (1) 翻译时,应该尽量使用简单的句子结构和语法,选择意义明确的词;
- (2) 检查翻译的内容是不是断章取义、是否词不达意;
- (3) 如果在源文件中使用了缩写词,检查缩写词在第一次出现的时候是否正确地标出了它的全称,以使用户能够明白其含义。这样,在其后的文本中即使一直用缩写词来表示,也没有关系;
- (4) 检查标点符号、货币、计量单位等是否符合当地习惯,包括显示格式。

在国际化基础上,本地化过程就相对简单。如果国际化没有做好,翻译和本地化的过程就会变为一个相当冗长的过程,其中包括将屏幕、对话框等重新设定;而且需要重建在线文件;图像和插图也可能需要更改。最后,计算机程序还可能需要做出某些修改去适应那些使用双字节字符的语言。

2. 目标语言文化的检查

翻译的时候要照顾到目标语言的文化心理,例如,对于不符合本地文化和软件产品的措辞,需要纠正。软件展示给用户的首先就是它的用户界面,所以 UI 的重要性是不言而喻的,这同样体现在本地化测试上。如果在 UI 上出现错误甚至有歧义的地方,会让用户对该软件产生不信任的心理,这些都是应该坚决杜绝的。再则,由于不同民族,不同信仰之间的差别,各民族在色彩、禁忌和其他习惯上也有很多区别。测试人员在软件本地化的翻译验证工作中,不仅要找出错误,而且应该找出不符合民族习惯的地方,能够提出有建设性的修改方案。

软件相关宣传品的处理,也要把握好当地的风俗习惯,留意包装的规格和颜色。例如,日本人比较忌讳数字“4”,就连 4 个一组包装的产品都不容易卖出去。而美国人不太喜欢鲜艳的红色,那么宣传资料和包装纸就应该尽量避免大红的颜色。

对于一些涉及文化方面的内容,最好能用本民族中相应的内容来替换。如中国人以红色为喜庆的颜色,遇到节日,网站采用大量红色,而且在股票交易网站上,股票价格上涨为红色,股票价格下跌为绿色,正好和西方相反,在美国股票价格上涨为绿色,股票价格下跌为红色。在西方,绿色象征安全,而红色意味着问题严重,如十字路口的红绿灯。再比如,安全警戒等级的最低等级是绿色(没问题),然后是黄色、橙色,最高等级是红色。作为国际化的软件产品,颜色应该是可以定制的,在进行本地化时,根据当地习俗将颜色重新设置,改变颜色的默认值。

3. 特殊符号

把一种语言翻译成另一种语言,同时还要注意目标语言的特殊符号,比如标点符号、货币

符号以及该目标语言所特有的其他符号。英语中的标点符号和亚洲语言的标点符号不太相同,英文的句号是一个圆点(单字节符号),而汉语和日语的句号都是一个小圆圈(双字节符号);汉语中的标点符号是比较完备的,英文中通常用斜体表示书名,汉字则用《》标识书名。

8.3 本地化测试的技术问题

完成了语言的转换,对于整个本地化过程来说,只是完成了第一阶段。要使该软件真正投入使用,还有很多技术方面的问题有待解决,主要有:

- (1) 数据格式。
- (2) 页面显示和布局。
- (3) 配置和兼容性问题。

8.3.1 数据格式

数字、货币和日期等的表达方法在不同的国家其格式也是不尽相同的,所以在对软件本地化时,也应该特别注意这些方面的问题,考虑到本地化格式的要求,否则就有可能出现错误。幸运的是,今天可以使用标准 APIs(比如微软、Sun 提供的)来处理这类转换的问题。如果是由自己设计的显示方式或模式,就必须很好地设计其变量含义和处理方式、数据存储方式等去适应这种显示的要求。

在程序设计、编程时,可以通过一些特殊的函数来处理不同语言的数据格式。例如,使用自定义函数 `LocLongdate()`, `LocShortdate()`, `LocTime()`, `LocNumberFormat()` 等替换原来的 `date()` 函数,来处理日期的完整表示、简写、数字等不同的显示格式。下面将通过一些具体的例子来介绍不同地方数字、货币和日期等不同的表达格式。

1. 数字

很多欧洲语言使用逗号而不是小数点来表示千位,有的则使用句号或空格代替逗号。所以,本地化的软件也必须注意这个问题,如若不然,有可能一个顾客存入 5000 欧元,而却只能取出 5 美元。比如,同一个数字(7582)在美国、意大利和瑞士有三种不同的表达方式:

美国: 7,582

意大利: 7.582

瑞士: 7 582

2. 货币

除了数字转换外,几乎每一个国家都有表示本国货币的符号,这些符号出现在金额的前后也各不相同。如果一个金融类的应用软件把本该用 表示的地方,用了 \$,后果将是不堪设想的。

美国: Dollar \$ 或 US\$。

英国: Pound £。

日本: Yen ¥。

欧洲: 欧元 。

中国: 人民币 。

3. 时间

各国时间的习惯表达方式也总是不一样的,美国习惯上使用 12 小时来表达时间,而欧洲国家使用 24 小时模式来表达时间。如晚上 10:45,在不同的国家有不同的表示。

- (1) 美国: 10: 45PM。
- (2) 德国: 22. 45。
- (3) 加拿大法裔: 22 h 45。

而且美国是 12:00am~11:59am, 12:00pm~11:59pm,没有 0:10am 或 0:30pm,相当于 12、1、2、…、10、11 这样的顺序。

4. 日期格式

同样,不同国家的日期显示格式也不是一致的。美国的标准是 MM/DD/YY 来显示月、日、年,也有很多不同的分割符号(如“/”和“-”);欧洲(除少数例外)的标准是日、月、年(DD/MM/YY);中国的标准则是年、月、日。下面以 2003 年 2 月 14 日为例来说明。

- (1) 美国: 2/14/2003。
- (2) 英国: 14. 2. 2003。
- (3) 中国: 2003/2/14。

即使是一个星期的起始天各国也不相同,如美国,一个星期的第一天是星期天;然而,法国的日历第一天都以星期一开头。

再看一个具体的例子,一个英文日期,如“7/22, 003”或“7-22-2003”,本地化为中文版本后,日期显示变为“7 月. 12, 2003”,显然不正确,其正确的中文显示应该是“2003 年 7 月 22 日”。

现在来了解一下正确的编码。从编码中可以看到在本地化的时候,有时必须应用自定义函数 LocLongDate()来解决日期显示的问题。这就要求本地化测试人员不只是发现问题,还要站在更高的层次来分析问题,并提出解决问题的建议。在 Java 里比较简单,有 java. util. Locale 类,日期格式化可以表示为:

```
SimpleDateFormat("",Locale.SIMPLIFIED_CHINESE);
```

根据语言版本取完整日期格式的处理函数(以下程序设计语言为 PHP 语言):

```
function LocLongDate( $ UnixTime, $ RegionID, $ DisplayWeek = "Yes")
{
    global $ glbRegion;
    ...

    InternationInit();
    if (!IsExistRegionID( $ RegionID))
        $ RegionID = $ glbDefaultRegionID;           //取得本地区域代码
    if ("". $ glbRegion[ $ RegionID][LONGDATEFORMAT] == "")    //如果是长日期型
        $ glbRegion[ $ RegionID][LONGDATEFORMAT] = "WWW, MMM d, yyyy";
    $ strFormat = FormatLocToFormatPhp( $ glbRegion[ $ RegionID][LONGDATEFORMAT]);
    if( $ DisplayWeek == "NoWeek")           //处理日期格式
    {
        $ strFormat = eregi_replace("l", "", $ strFormat);
        $ strFormat = ereg_replace("^, ", "", $ strFormat);
    }
    $ LongDateString = date( $ strFormat, $ UnixTime);
```

```

        if (strstr(strtolower( $ glbRegion[ $ RegionID][LONGDATEFORMAT]), "www"))
            $ LongDateString = str_replace(date("l", $ UnixTime), $ ARR_FULLWEEKDAY[date("w",
$ UnixTime)], $ LongDateString); //获得星期显示字符串
        if (strstr(strtolower( $ glbRegion[ $ RegionID][LONGDATEFORMAT]), "mmm"))
            $ LongDateString = str_replace(date("F", $ UnixTime), ARR_FULLMONTH[ date("n",
$ UnixTime) - 1 ], $ LongDateString); //获得日期显示字符串
        return $ LongDateString;
    }

```

5. 度量衡的单位

美国以外的很多国家使用公制度量系统,因此,国际化的软件必须能够解决公制度量单位的问题。度量衡的单位在工程和科学软件中尤为敏感,如果在转换的过程中出现错误,后果将不堪设想。所以在转换英式度量单位制和公制度量单位时,要倍加小心。这更是本地化测试所不容忽略的问题。

6. 索引和排序

英文排序和索引习惯上按照字母的顺序来编排,但是对于一些非字母文字的国家,如亚洲很多国家来说,这种方法就不适用了,如汉字就有按拼音、部首和笔画等不同的索引方法;即使是使用字母文字的国家,他们的排序方法和英文也是有很大出入的,比如瑞典语,它的字母比英文字母多三个,在索引排序时也应加以考虑。所以,在本地化软件时,应该根据不同国家和地区的语言习惯分别加以考虑,在进行本地化测试的时候更应该仔细核对这些问题,如把英文软件本地化为瑞典版本中,用来排序的有 29 个字母,在字母 A, B, C, ..., X, Y, Z 后会增加几个特殊的字母——瑞典语中的三个字母,即 Ä, Å, Ö。从代码中可以看到,它分别用了(\$ Index == (RawUrlDecode("%C5"))、(\$ Index == (rawurldecode("%C4"))和(\$ Index == (rawurldecode("%D6"))来表示这几个字母。

```

if (GetLanguageIDFromUrl(BrandName()) == 13)
{
    if ( $ Index == (rawurldecode("%C5")))
        echo "<b> &Aring;</b> &nbsp;&nbsp;&nbsp;";
    else
        print("< A HREF = \"javascript:GetAddressByIndex(document.FormDeleteAddress, '". $ strSortField.'',
'\". '&Aring;'. \"')\">\". '&Aring;'. \"</A> &nbsp;&nbsp;&nbsp;\n");
    if ( $ Index == (rawurldecode("%C4")))
        echo "<b> &Auml;</b> &nbsp;&nbsp;&nbsp;";
    else
        print("< A HREF = \"javascript:GetAddressByIndex(document.FormDeleteAddress, '". $ strSortField.'',
'\". '&Auml;'. \"')\">\". '&Auml;'. \"</A> &nbsp;&nbsp;&nbsp;\n");
    if ( $ Index == (rawurldecode("%D6")))
        echo "<b> &Ouml;</b> &nbsp;&nbsp;&nbsp;";
    else
        print("< A HREF = \"javascript:GetAddressByIndex(document.FormDeleteAddress, '". $ strSortField.'',
'\". '&Ouml;'. \"')\">\". '&Ouml;'. \"</A> &nbsp;&nbsp;&nbsp;\n");
}

```

7. 姓名格式

英文的姓名格式是名在前,姓在后,姓名之间还需空一格,而亚洲人的姓名格式通常是姓前名后,而且中间无须空格。由于这个区别,在做本地化测试的时候,一定要确保受影响的部

分都做了相应的改动,否则会导致显示和查找的时候产生错误。在测试的时候如果发现此类错误,可以建议编码人员根据不同国家和地区的语言习惯考虑姓、名以及全名之间的关系,自定义一些函数来处理此类问题。

这里定义了函数 `GetFullNameforMultipleLanguage`,来帮助我们识别姓名的类型,从而选取正确的显示格式。

```
function GetFullNameforMultipleLanguage( $ FirstName, $ LastName = "" ) //根据语言版本取得相应的
//姓名格式
{
    //如果全名中包含英文大写字符 A~Z 或小写字符 a~z,则保留名和姓之间的空格
    $ strFullName = trim( $ FirstName. " ". $ LastName );
    if( eregi("[a-z]", $ strFullName[0])) //如果姓名是英文字符,则立刻返回其值
    return $ strFullName;
    //如果当前语言是繁体中文,则删除其中的空格
    $ LanguageID = GetCookie("CK_LanguageID_".GetSiteConfig("SiteID"));
    if( intval( $ LanguageID) == 0)
    $ LanguageID = GetLanguageIDFromUrl( BrandName());
    if ( $ LanguageID == 4 || $ LanguageID == 5) // 针对繁体中文和日语
    return ereg_replace(" * ", "", $ strFullName);

    if( eregi("[a-z]", $ strFullName))
    return trim( $ strFullName);
    else
    return ereg_replace(" * ", "", $ strFullName);
}
```

8. 复数问题

生成复数的规则因语言的不同而有差异。即使在英语中,复数的规则也并不是始终如一的,如“bed”的复数是“beds”,而“leaf”的复数却不是“leafs”,以下例子说明了复数的问题。如:

```
" %d program %s searched"
```

和

```
" %d file %s searched".
```

如果 %d 大于 1, %s 将把“s”插入到该单词中去从而组成其复数形式,则该信息显示格式如下:

```
"1 program searched" and "1 file searched"
```

或者

```
"3 programs searched" and "3 files searched."
```

在英语中,这样编码是没有问题的,但是对于德语和多数其他欧洲语言,它们的复数规则却不是这样的,如:

```
program = programma
programs = programma's
file = bestand
files = bestanden
```

在做本地化测试的时候,特别要注意这些地方是否被充分地考虑并做了适当的修改。

PHP 支持国际化和本地化特性

PHP 语言从 PHP5 版本更好地支持国际化和本地化的需要。例如,定义了一个 locale 类,覆盖了语言、区域、姓名等;定义了一个 DateTimeZone 类来处理全球主要国家的时区。可以在 hp.ini 文件中设定区域和时区,即 date.timezone 设置为特定时区(如 Etc/GMT-8 或 Asia/Beijing),也可以在 PHP 程序里用 date_default_timezone_set() 设置。

使用 setlocale(string category, string locale)、locale_set_default(string \$name)、date_default_timezone_set() 设置本地化环境。然后使用如 money_format()、number_format() 和 strftime() 以及 localeconv() 等函数,就能获得货币、数字、时间等格式化的数据。例如:

```
< php
date_default_timezone_set('Europe/Helsinki');
setlocale(LC_ALL, 'nl_NL');
echo strftime("%A %e %B %Y", mktime(0, 0, 0, 12, 22, 1978));
>

Locale {
    /* Methods */
    static string acceptFromHttp (string $header)
    static string composeLocale (array $subtags)
    static bool filterMatches (string $langtag , string $locale)
    static array getAllVariants (string $locale)
    static string getDefault (void)
    static string getDisplayLanguage (string $locale [, string $in_locale ])
    static string getDisplayName (string $locale [, string $in_locale ])
    static string getDisplayRegion (string $locale [, string $in_locale ])
    static string getDisplayScript (string $locale [, string $in_locale ])
    static string getDisplayVariant (string $locale [, string $in_locale ])
    static array getKeywords (string $locale)
    static string getPrimaryLanguage (string $locale)
    static string getRegion (string $locale)
    static string getScript (string $locale)
    static string lookup (array $langtag , string $locale)
    static array parseLocale (string $locale)
    static bool setDefault (string $locale)
}

DateTimeZone {
    /* Constants */
    const integer DateTimeZone::AFRICA = 1;
    const integer DateTimeZone::AMERICA = 2;
    const integer DateTimeZone::ANTARCTICA = 4;
    const integer DateTimeZone::ARCTIC = 8;
    const integer DateTimeZone::ASIA = 16;
    const integer DateTimeZone::ATLANTIC = 32;
    const integer DateTimeZone::AUSTRALIA = 64;
    const integer DateTimeZone::EUROPE = 128;
    const integer DateTimeZone::INDIAN = 256;
    const integer DateTimeZone::PACIFIC = 512;
```

```

const integer DateTimeZone::UTC = 1024;
const integer DateTimeZone::ALL = 2047;
const integer DateTimeZone::ALL_WITH_BC = 4095;
const integer DateTimeZone::PER_COUNTRY = 4096;
/* Methods */
public construct (string $ timezone)
public array getLocation (void)
public string getName (void)
public int getOffset (DateTime $ datetime)
public array getTransitions ([ int $ timestamp_begin [, int $ timestamp_end ]])
public static array listAbbreviations (void)
public static array listIdentifiers ([ int $ what = DateTimeZone::ALL [, string $ country =
NULL ]])
}

```

8.3.2 页面显示和布局

在有些本地化软件中,有时会发现乱码的问题,这是由于没有设置相应的本地化字符集或字符编码方式不支持本地化语言所致,不同的浏览器或邮件接收软件的编码解码方式不同,解决这类问题的方法如下。

(1) 开发本地化时应用自定义函数 GetCurCharSet()。

```

function GetCurCharSet()                                // bind charset to language
{
    $ CharSet = "iso - 8859 - 1";                        //标准字符集
    $ LanguageID = GetCurLanguageID();

    if ( $ LanguageID == 3)    $ CharSet = "gb2312";      //简体中文版
    if ( $ LanguageID == 4)    $ CharSet = "big5";        //繁体中文版
    if ( $ LanguageID == 5)    $ CharSet = "shift_jis";   //日文版
    if ( $ LanguageID == 6)    $ CharSet = "euc - kr";    //韩文版
    return $ CharSet;
}

```

这个函数中调用了另一个自定义函数 GetCurLanguageID(),而函数的值是通过本机的 Cookies 取到的,这样可以通过 GetCurCharSet()调用该函数来判断采用相应的字符集。

```

function GetCurLanguageID()                            //取得相应的语言版本
{
    $ LanguageID = GetCookie("CK_LanguageID_".GetSiteConfig("SiteID"));
    if(intval( $ LanguageID) == 0)
        $ LanguageID = GetLanguageIDFromUrl(BrandName());

    return intval( $ LanguageID);
}

```

(2) 针对不同的浏览器采取不同的解码方法。

```

Function Preview(form) {
    var NS4 = (document.layers && !dom) 1:0;
    var NS6 = (navigator.vendor == ("Netscape6") || navigator.product == ("Gecko"));
}

```

```
var message;
var re = /\ + /g;
if (NS4 > 0)
    message = escape(form.welmsg.value);
else if(NS6 > 0)
    message = form.welmsg.value;
else
    message = form.welmsg.innerHTML;

if(message.indexOf(" ")!= -1){
    var wmessage = message.replace(re, "% 2B");
}
else wmessage = message;
form.AT.value = "Submit";
form.preview.value = "true";

var sTemp;
if (NS4 > 0 || NS6 > 0)
    sTemp = form.welmsg.value;
else
    sTemp = form.welmsg.innerHTML;
if(getStrLength(sTemp)>128)
{
    alert("欢迎消息不能超过 128 个字符.");    //给出警告提示
    form.welmsg.focus();
}
else
    window.open("< = PersonalLobbyPath( ) > index.php username = < = rawurlencode( str_
replace("&quot;","\"",$ repinfo["UserName"])) > &preview = true&welmsg = " + wmessage);
}
```

由于源代码没有充分考虑到国际化(I18N)版本的要求,很多软件本地化之后在页面的外观上会出现一些不尽如人意的地方。如没有翻译的字段、对齐问题、大小写问题、文字遮挡图像问题、显示乱码问题等。这些有表格设定所产生的问题,也有未考虑翻译后的文字扩展而产生的设计问题。测试人员应及时指出这些错误,让开发人员尽快修改。

8.3.3 配置和兼容性问题

测试本地化软件的时候,其配置和兼容性也是必须考虑的问题。配置性包括键盘布局设计、打印机配置等。软件可能会用到的任何外设都要在平台配置和兼容性测试的等价区间中考虑。兼容性包括与硬件的兼容性、与上一版本的数据兼容及与其他本地化软件的兼容性等。

1. 数据库问题

软件本地化同时也涉及数据库的改动,比如由于文本的 maxlen 属性只限制输入字符而非字节长度或非 ASCII 码,特别是多字节字符解析成 NCR 形式(&# dddd;),导致输入的字符长度超出数据库字段宽度,这就是由数据库而产生的问题。

在本地化过程中,应视情况而定。比如可以在输入页面提交之前,检测输入字符的宽度是

否超长或显示数据库操作错误。如：

```
...
<form name = AddVis>
<input name = "V_Name" type = text value = "">
<input type = submit value = "ok" onsubmit = "javascript:checkinput()">
...

function getStrLength(StrTemp)           //取字符长度
{
    var strInput = "" + StrTemp;
    var i,sum;
    sum = 0;
    //alert("string lengh = " + strInput.length);
    for(i = 0;i < strInput.length;i++)
    {
        if((strInput.charCodeAt(i) >= 0)&&(strInput.charCodeAt(i) <= 255))
            sum = sum + 1;
        else
            sum = sum + 2;
    }
    //alert("actual lengh = " + sum);
    return sum;
}
function checkinput()                   //检查输入字符的长度
{
    if(getStrLength(document.AddVis.V_Name.value) > 64)
    {
        alert("The input string must be less than 64. ");           //给出警告提示
        document.AddVis.V_Name.focus();
        return false;
    }
}
```

2. 热键

在做本地化测试的时候,还有一个不能忽略的问题——热键问题。许多程序都为不同的命令设置了热键(键盘快捷方式)。比如,在微软的 Word 中,可以按 Ctrl+F 键打开“查找”对话框。热键 Ctrl+F 就是代替鼠标来选择 Word“编辑”菜单中“查找”命令的简捷方式。通常,文字被翻译之后,原来的热键很可能不再适用,需要为翻译过的文本设定新的热键,比如,当“Close”被翻译成德语“Schließen”之后,原有的热键 Alt+C 也应该相应地变为 Alt+S。新的热键应该和本地操作系统环境相匹配,确保所有的热键都是唯一的。不过中国、日本和韩国的版本,都沿用英文原有的热键,所以本地化之后不存在这个问题。

此外,还有很多应该注意的技术问题,如:对于欧洲语言的本地化,还有大小写字母转换的问题、连字符连接规则、键盘的问题等。对于有些国家的本地化,例如希伯莱文和阿拉伯文还要考虑文字显示方向的问题等。

8.4 本地化的功能测试

软件本地化是一个再创造的过程,不仅包括翻译人员的劳动、技术人员的再加工,而且包括测试人员的层层把关。软件本地化之后,把它当作新的版本来对待,针对改动的地方进行充

分的测试,特别是前面介绍的翻译问题和技术问题,并完成相应的回归功能测试。

任何一件产品,人们最关心的还是它所能提供的服务,所以功能的实现总是很重要的。要验证一个软件是否被正确地本地化,要在相对真实的环境下对软件所有功能进行测试。关于本地化软件的功能测试,可以和源语言版本相对比来进行测试。此外,还要注意是否能够正确地输入目标语言、输入之后是否能够正确显示等。

1. 联机文档的功能测试

就像打印好的文档一样,测试人员应该验证任何一个联机文档的有效性、可用性。本地化软件测试人员应该对它们进行功能测试,以确保它们能够正常工作,并且与目标市场的要求一致。

不论是 PDF 还是 HTML 格式的联机文档,都应该在目标语言的操作系统下测试,确保其功能能够实现,字符能够正确显示,一般来说,主要检测文件的以下几方面。

- (1) 与目标语言操作系统的兼容性;
- (2) 字体和图形能够正确显示;
- (3) 与本地化的 Acrobat Reader 版本和 HTML 浏览器兼容;
- (4) 超链接的正常跳转。

2. 页面内容和图片

在页面测试时要时时提醒自己,HTML 页面上有些文字不是一眼就能看到的,如:

- (1) 显示在浏览器界面顶部的页面的标题。
- (2) 图片的标题,当图片正在下载或者用户鼠标指向该图形时所显示的 Alt 属性。
- (3) 超级链接的标题。

确保这些内容也被本地化处理了。

3. Web 链接和高级选项

测试员需要关注页面上的超级链接未被本地化的部分或者链接到其他未被本地化的站点上去,否则应用目标语言在这些链接旁给出提示,指出这些站点是源语言的。网站日益注重提供更多的动画效果,如 Flash,需要针对这些动画效果进行测试。还要检查浏览器的一些高级选项,如 JavaScript 脚本和 ActiveX Applets 等相关的设置,以了解应用软件是否受到影响。

小结

国际化是本地化的基础和前提,本地化是国际化向特定本地语言环境的转换,其理想的状态是,源语言版本要按照国际化版本的要求去做,本地化本身不应该再给该软件增加新的功能缺陷。然而,如果该软件没有充分地国际化,在本地化过程中就很有可能会产生新的功能性方面的问题,包括功能调用出错、输入和输出问题等。

翻译仅是软件本地化的一部分工作,软件本地化实际是一项技术工作,要处理字符集问题、数据格式、页面显示和布局、配置和兼容性问题等。在测试过程中,应该特别注意这方面的问题,特别是时区、日期、时间、货币、度量衡、姓名、复数等的处理和显示。为确保本地化后软件产品的质量,本地化的产品应该在配置有目标语言操作系统的计算机上进行测试。本地化的翻译人员、培训人员和技术支持人员也最好参与到本地化的测试中来,以保证该测试的全面性和完整性。

思考题

1. 为什么要进行软件本地化?
2. 软件本地化和软件国际化有什么关系?
3. 为什么说软件本地化不等同于翻译?
4. 软件本地化测试中应该着重于哪些方面?
5. 进行软件本地化测试是否必须通晓该目标语言? 为什么?
6. 假设需要测试某一软件的日语本地化版本,请问需要做哪些方面的准备? 请一一列举。